

Data Mining

Lab - 1

¥@\$# Kakadiya | 23010101121 | 4/6/2025

Introduction to Pandas Library Function:

Step-1 Import the pandas Libraries

```
In [1]: import pandas as pd
```

Step-2 Import the dataset from this:....

```
In [ ]:
```

Step-3 Read csv or excel File

```
In [2]: df = pd.read_csv("../data/titanic.csv")
```

Step-4 Print Data from csv or excel File

```
In [3]: df
```

Out[3]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	

891 rows × 12 columns



Step-5 See the First 10 Rows

In [4]:

```
df.head(10)
```

Out[4]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emba
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	

Step-6 See the Last 10 Rows

In [5]:

df.tail(10)

Out[5]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Er
881	882	0	3	Markun, Mr. Johann	male	33.0	0	0	349257	7.8958	NaN	
882	883	0	3	Dahlberg, Miss. Gerda Ulrika	female	22.0	0	0	7552	10.5167	NaN	
883	884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A./SOTON 34068	10.5000	NaN	
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OQ 392076	7.0500	NaN	
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.1250	NaN	
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	

Step-7 Data type of each columns

In [6]: df.dtypes

Out[6]: PassengerId int64
Survived int64
Pclass int64
Name object
Sex object
Age float64
SibSp int64
Parch int64
Ticket object
Fare float64
Cabin object
Embarked object
dtype: object

Step-8 Display Summary Information

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null    int64
1   Survived        891 non-null    int64
2   Pclass          891 non-null    int64
3   Name            891 non-null    object
4   Sex             891 non-null    object
5   Age            714 non-null    float64
6   SibSp           891 non-null    int64
7   Parch          891 non-null    int64
8   Ticket          891 non-null    object
9   Fare           891 non-null    float64
10  Cabin           204 non-null    object
11  Embarked        889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Step-9 Access a specific column

```
In [8]: df["Name"]
```

```
Out[8]: 0          Braund, Mr. Owen Harris
1  Cumings, Mrs. John Bradley (Florence Briggs Th...
2          Heikkinen, Miss. Laina
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)
4          Allen, Mr. William Henry
...
886          Montvila, Rev. Juozas
887          Graham, Miss. Margaret Edith
888  Johnston, Miss. Catherine Helen "Carrie"
889          Behr, Mr. Karl Howell
890          Dooley, Mr. Patrick
Name: Name, Length: 891, dtype: object
```

Step-10 Access rows by their integer location

```
In [9]: df.iloc[7]
```

```
Out[9]: PassengerId     8
Survived              0
Pclass               3
Name      Palsson, Master. Gosta Leonard
Sex              male
Age              2.0
SibSp             3
Parch             1
Ticket          349909
Fare            21.075
Cabin            NaN
Embarked          S
Name: 7, dtype: object
```

Step-11 Delete a specific Column

```
In [10]: # df=df.drop(columns=['Cabin'])
df.drop("Cabin", axis=1, inplace=True)
df
```

Out[10]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S
...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	Q

891 rows × 11 columns

Step-12 Create a new Column

```
In [11]: df["isName"] = ~df["Name"].isnull()
df
```

Out[11]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S
...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	Q

891 rows × 12 columns



Step-13 Perform Condition Selection on DataFrame

In [12]:

```
df[(df["Pclass"] > 1) & (df["isName"] == True)]
```

Out[12]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	C
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	S
...	...	...	...	...	...	...	...	...	...	...	...
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OQ 392076	7.0500	S
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.1250	C
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	S
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	C

675 rows × 12 columns



Step-14 Compute the sum of value

In [13]:

```
df["Fare"].sum()
```

Out[13]: 28693.9493

Step-15 Compute the mean of value

In [14]:

```
df["Fare"].mean()
```

Out[14]: 32.204207968574636

Step-16 Count non-null value (column)

```
In [15]: df.count()
```

```
Out[15]: PassengerId      891  
Survived      891  
Pclass      891  
Name      891  
Sex      891  
Age      714  
SibSp      891  
Parch      891  
Ticket      891  
Fare      891  
Embarked      889  
isName      891  
dtype: int64
```

Step-17 Find Minimum or Maximum values

```
In [16]: print(df["Fare"].min())
```

```
print(df["Fare"].max())
```

```
0.0
```

```
512.3292
```

Data Mining

Lab - 2

¥@\$# Kakadiya | 23010101121 |
11/6/2025

Numpy & Perform Data Exploration with Pandas

Numpy

1. NumPy (Numerical Python) is a powerful open-source library in Python used for numerical and scientific computing.
2. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on them efficiently.
3. NumPy is highly optimized and written in C, making it much faster than using regular Python lists for numerical operations.
4. It serves as the foundation for many other Python libraries in data science and machine learning, like pandas, TensorFlow, and scikit-learn.
5. With features like broadcasting, vectorization, and integration with C/C++ code, NumPy allows for cleaner and faster code in numerical computations.

Step 1. Import the Numpy library

```
In [1]: import numpy as np  
  
np.__version__
```

```
Out[1]: '1.26.4'
```

Step 2. Create a 1D array of numbers

```
In [2]: arr = np.array([1,2,3,4,5,6,7])  
arr
```

```
Out[2]: array([1, 2, 3, 4, 5, 6, 7])
```

```
In [3]: arr=np.arange(1,8,2)  
arr
```

```
Out[3]: array([1, 3, 5, 7])
```

```
In [4]: arr2d=np.array([[1,2],[3,4],[5,6]])  
arr2d
```

```
Out[4]: array([[1, 2],  
              [3, 4],  
              [5, 6]])
```

Step 3. Reshape 1D to 2D Array

```
In [5]: arrReshape=np.arange(8).reshape(2,4)  
  
print(arrReshape)  
print(arrReshape.dtype)  
print(arrReshape.size)  
print(type(arrReshape))  
print(type(arrReshape[0,1]))  
print(arrReshape.ndim)  
  
[[0 1 2 3]  
 [4 5 6 7]]  
int32  
8  
<class 'numpy.ndarray'>  
<class 'numpy.int32'>  
2
```

Step 4. Create a Linspace array

```
In [6]: arr=np.linspace(0,10,5)  
arr
```

```
Out[6]: array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

Step 5. Create a Random Numbered Array

```
In [7]: arr=np.random.rand(7)  
arr
```

```
Out[7]: array([0.89131114, 0.52252606, 0.70993858, 0.77093598, 0.35612336,  
              0.52889868, 0.18022875])
```

Step 6. Create a Random Integer Array

```
In [8]: arr=np.random.randint(1,7,size=7)  
arr
```

```
Out[8]: array([4, 4, 3, 3, 4, 3, 5])
```

Step 7. Create a 1D Array and get Max,Min,ArgMax,ArgMin

```
In [9]: arr=np.random.randint(1,700,size=7)  
arr
```

```
Out[9]: array([ 93, 504, 329, 280, 625, 658, 120])
```

```
In [10]: arr.max()
```

```
Out[10]: 658
```

```
In [11]: arr.min()
```

```
Out[11]: 93
```

```
In [12]: arr.argmax()
```

```
Out[12]: 5
```

```
In [13]: arr.argmin()
```

```
Out[13]: 0
```

```
In [14]: arr.mean()
```

```
Out[14]: 372.7142857142857
```

Step 8. Indexing in 1D Array

```
In [15]: arr=np.random.randint(1,700,size=7)
print(arr)
print(arr[6])
print(arr[-1])
```

```
[391 421 217 331 687 574 176]
176
176
```

Step 9. Indexing in 2D Array

```
In [16]: arr2d=np.array([[1,2],[3,4],[5,6]])
print(arr2d[1,0])
print(arr2d[-1,0])
```

```
3
5
```

Step 10. Conditional Selection

```
In [17]: arr=np.random.randint(1,700,size=7)
print(arr)
arr[arr>3]
```

```
[681 451 102 646 394 644 353]
```

```
Out[17]: array([681, 451, 102, 646, 394, 644, 353])
```

```
In [18]: arr2d=np.array([[1,2],[3,4],[5,6]])
print(arr2d)
```

```
[[1 2]
 [3 4]
 [5 6]]
```

🔥 You did it! 10 exercises down — you're on fire! 🔥

Pandas

Step 1. Import the necessary libraries

```
In [19]: import pandas as pd
pd.__version__
```

Out[19]: '2.2.2'

Step 2. Import the dataset from this [address](https://raw.githubusercontent.com/justmarkham/DAT8/master/data/u.user).

```
In [20]: df=pd.read_csv('https://raw.githubusercontent.com/justmarkham/DAT8/master/data/u.user',sep='|')
df
```

```
Out[20]:
```

	user_id	age	gender	occupation	zip_code
0	1	24	M	technician	85711
1	2	53	F	other	94043
2	3	23	M	writer	32067
3	4	24	M	technician	43537
4	5	33	F	other	15213
...	...	...	...	...	...
938	939	26	F	student	33319
939	940	32	M	administrator	02215
940	941	20	M	student	97229
941	942	48	F	librarian	78209
942	943	22	M	student	77841

943 rows × 5 columns

Step 3. Assign it to a variable called users and use the 'user_id' as index

```
In [21]: users=df.set_index('user_id')
users
```

```
Out[21]:
```

	age	gender	occupation	zip_code
user_id				
1	24	M	technician	85711
2	53	F	other	94043
3	23	M	writer	32067
4	24	M	technician	43537
5	33	F	other	15213
...	...	...	...	...
939	26	F	student	33319
940	32	M	administrator	02215
941	20	M	student	97229
942	48	F	librarian	78209
943	22	M	student	77841

943 rows × 4 columns

Step 4. See the first 25 entries

```
In [22]: users.head(25)
```

```
Out[22]:
```

	age	gender	occupation	zip_code
user_id				
1	24	M	technician	85711
2	53	F	other	94043
3	23	M	writer	32067
4	24	M	technician	43537
5	33	F	other	15213
6	42	M	executive	98101
7	57	M	administrator	91344
8	36	M	administrator	05201
9	29	M	student	01002
10	53	M	lawyer	90703
11	39	F	other	30329
12	28	F	other	06405
13	47	M	educator	29206
14	45	M	scientist	55106
15	49	F	educator	97301
16	21	M	entertainment	10309
17	30	M	programmer	06355
18	35	F	other	37212
19	40	M	librarian	02138
20	42	F	homemaker	95660
21	26	M	writer	30068
22	25	M	writer	40206
23	30	F	artist	48197
24	21	F	artist	94533
25	39	M	engineer	55107

Step 5. See the last 10 entries

```
In [23]: users.tail(10)
```

Out[23]:

	age	gender	occupation	zip_code
--	-----	--------	------------	----------

user_id				
934	61	M	engineer	22902
935	42	M	doctor	66221
936	24	M	other	32789
937	48	M	educator	98072
938	38	F	technician	55038
939	26	F	student	33319
940	32	M	administrator	02215
941	20	M	student	97229
942	48	F	librarian	78209
943	22	M	student	77841

Step 6. What is the number of observations in the dataset?

In [24]: `users.shape[0]`

Out[24]: 943

Step 7. What is the number of columns in the dataset?

In [25]: `users.shape[1]`

Out[25]: 4

In [26]: `users.shape`

Out[26]: (943, 4)

Step 8. Print the name of all the columns.

In [27]: `users.columns`

Out[27]: Index(['age', 'gender', 'occupation', 'zip_code'], dtype='object')

Step 9. How is the dataset indexed?

In [28]: `users.index`

Out[28]: Index([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, ...
934, 935, 936, 937, 938, 939, 940, 941, 942, 943],
dtype='int64', name='user_id', length=943)

In [29]: `users.index.name`

Out[29]: 'user_id'

Step 10. What is the data type of each column?

In [30]: `users.dtypes`

```
Out[30]: age          int64
gender      object
occupation  object
zip_code    object
dtype: object
```

Step 11. Print only the occupation column

```
In [31]: users['occupation']
```

```
Out[31]: user_id
1          technician
2             other
3            writer
4          technician
5             other
...
939         student
940  administrator
941         student
942        librarian
943         student
Name: occupation, Length: 943, dtype: object
```

Step 12. How many different occupations are in this dataset?

```
In [32]: users['occupation'].nunique()
```

```
Out[32]: 21
```

```
In [33]: users['occupation'].unique()
```

```
Out[33]: array(['technician', 'other', 'writer', 'executive', 'administrator',
               'student', 'lawyer', 'educator', 'scientist', 'entertainment',
               'programmer', 'librarian', 'homemaker', 'artist', 'engineer',
               'marketing', 'none', 'healthcare', 'retired', 'salesman', 'doctor'],
              dtype=object)
```

Step 13. What is the most frequent occupation?

```
In [34]: users['occupation'].value_counts().head(1)
```

```
Out[34]: occupation
student    196
Name: count, dtype: int64
```

Step 14. Summarize the DataFrame.

```
In [35]: users.describe()
```

Out[35]:

age	
count	943.000000
mean	34.051962
std	12.192740
min	7.000000
25%	25.000000
50%	31.000000
75%	43.000000
max	73.000000

Step 15. Summarize all the columns

In [36]: `users.describe(include='all')`

Out[36]:

	age	gender	occupation	zip_code
count	943.000000	943	943	943
unique	NaN	2	21	795
top	NaN	M	student	55414
freq	NaN	670	196	9
mean	34.051962	NaN	NaN	NaN
std	12.192740	NaN	NaN	NaN
min	7.000000	NaN	NaN	NaN
25%	25.000000	NaN	NaN	NaN
50%	31.000000	NaN	NaN	NaN
75%	43.000000	NaN	NaN	NaN
max	73.000000	NaN	NaN	NaN

Step 16. Summarize only the occupation column

In [37]: `users['occupation'].describe()`

Out[37]:

count	943
unique	21
top	student
freq	196

Name: occupation, dtype: object

Step 17. What is the mean age of users?

In [38]: `users['age'].mean()`

Out[38]: 34.05196182396607

Step 18. What is the age with least occurrence?

In [39]: `users['age'].value_counts().tail()`

```
Out[39]: age
7      1
66     1
11     1
10     1
73     1
Name: count, dtype: int64
```

You're not just learning, you're mastering it. Keep aiming higher! 🚀

Data Mining

Lab - 3

¥@\$# Kakadiya | 23010101121 |
18/6/2025

1) First, you need to read the titanic dataset from local disk and display first five records

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: df=pd.read_csv('../data/titanic.csv')
```

```
In [3]: df.head(5)
```

```
Out[3]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	

2) Identify Nominal, Ordinal, Binary and Numeric attributes from data sets and display all values.

```
In [4]: nominal=['Name','Ticket','Cabin']
ordinal=['Pclass','Embarked']
binary=['Survived','Sex']
numeric=['PassengerId','Fare','SibSp','Age','Parch']
print('Nominal:',nominal)
print('Ordinal:',ordinal)
print('Binary:',binary)
print('Numeric:',numeric)
```

```
Nominal: ['Name', 'Ticket', 'Cabin']
Ordinal: ['Pclass', 'Embarked']
Binary: ['Survived', 'Sex']
Numeric: ['PassengerId', 'Fare', 'SibSp', 'Age', 'Parch']
```

3) Identify symmetric and asymmetric binary attributes from data sets and display all values.

```
In [5]: symmetric=['Sex']
asymmetric=['Survived']
print('Symmetric:\n',df['Sex'].value_counts())
print('Asymmetric:\n',df['Survived'].value_counts())
```

```
Symmetric:
Sex
male      577
female    314
Name: count, dtype: int64
Asymmetric:
Survived
0      549
1      342
Name: count, dtype: int64
```

4) For each quantitative attribute, calculate its average, standard deviation, minimum, mode, range and maximum values.

```
In [6]: columns=['PassengerId','Survived','Pclass','Age','SibSp','Parch','Fare',];

for i in columns:
    print(i,': ')
    print('\tAverage: ', df[i].mean())
    print('\tStandard deviation: ', df[i].std())
    print('\tMinimum: ', df[i].min())
    print('\tMode: ', df[i].mode()[0])
    print('\tRange: ', df[i].max()-df[i].min())
    print('\tMax: ', df[i].max())
    print('-----')
```

```

PassengerId :
    Average:  446.0
    Standard deviation:  257.3538420152301
    Minimum:  1
    Mode:  1
    Range:  890
    Max:  891
-----
Survived :
    Average:  0.3838383838383838
    Standard deviation:  0.4865924542648585
    Minimum:  0
    Mode:  0
    Range:  1
    Max:  1
-----
Pclass :
    Average:  2.308641975308642
    Standard deviation:  0.8360712409770513
    Minimum:  1
    Mode:  3
    Range:  2
    Max:  3
-----
Age :
    Average:  29.69911764705882
    Standard deviation:  14.526497332334044
    Minimum:  0.42
    Mode:  24.0
    Range:  79.58
    Max:  80.0
-----
SibSp :
    Average:  0.5230078563411896
    Standard deviation:  1.1027434322934275
    Minimum:  0
    Mode:  0
    Range:  8
    Max:  8
-----
Parch :
    Average:  0.38159371492704824
    Standard deviation:  0.8060572211299559
    Minimum:  0
    Mode:  0
    Range:  6
    Max:  6
-----
Fare :
    Average:  32.204207968574636
    Standard deviation:  49.693428597180905
    Minimum:  0.0
    Mode:  8.05
    Range:  512.3292
    Max:  512.3292
-----

```

6) For the qualitative attribute (class), count the frequency for each of its distinct values.

```

In [7]: qualitative_attributes=['Pclass','Name','Ticket','Cabin','Embarked','Sex']
for i in qualitative_attributes:
    print(df[i].value_counts())
    print('-----')

```

```

Pclass
3      491
1      216
2      184
Name: count, dtype: int64
-----
Name
Braund, Mr. Owen Harris      1
Boulos, Mr. Hanna           1
Frolicher-Stehli, Mr. Maxmillian  1
Gilinski, Mr. Eliezer        1
Murdlin, Mr. Joseph          1
..
Kelly, Miss. Anna Katherine "Annie Kate"  1
McCoy, Mr. Bernard          1
Johnson, Mr. William Cahoone Jr  1
Keane, Miss. Nora A         1
Dooley, Mr. Patrick          1
Name: count, Length: 891, dtype: int64
-----
Ticket
347082      7
CA. 2343     7
1601         7
3101295     6
CA 2144      6
..
9234         1
19988        1
2693         1
PC 17612     1
370376       1
Name: count, Length: 681, dtype: int64
-----
Cabin
B96 B98      4
G6            4
C23 C25 C27  4
C22 C26      3
F33          3
..
E34          1
C7           1
C54          1
E36          1
C148         1
Name: count, Length: 147, dtype: int64
-----
Embarked
S      644
C      168
Q       77
Name: count, dtype: int64
-----
Sex
male      577
female    314
Name: count, dtype: int64
-----

```

7) It is also possible to display the summary for all the attributes simultaneously in a table using the `describe()` function. If an attribute is quantitative, it will display its mean, standard deviation and various quantiles (including minimum, median, and maximum) values. If an attribute is qualitative, it will display its number of unique values and the top (most frequent) values.

```
In [8]: print('Summary of numeric attributes:')
print(df.describe())
print()
print('-----')

print('Summary of all attributes:')
print(df.describe(include = 'all'))
print()
print('-----')

print('Summary of categorical attributes:')
print(df.describe(include = 'object'))
```

Summary of numeric attributes:

	PassengerId	Survived	Pclass	Age	SibSp \
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Summary of all attributes:

	PassengerId	Survived	Pclass	Name	Sex \
count	891.000000	891.000000	891.000000	891	891
unique	NaN	NaN	NaN	891	2
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male
freq	NaN	NaN	NaN	1	577
mean	446.000000	0.383838	2.308642	NaN	NaN
std	257.353842	0.486592	0.836071	NaN	NaN
min	1.000000	0.000000	1.000000	NaN	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN
max	891.000000	1.000000	3.000000	NaN	NaN

	Age	SibSp	Parch	Ticket	Fare	Cabin \
count	714.000000	891.000000	891.000000	891	891.000000	204
unique	NaN	NaN	NaN	681	NaN	147
top	NaN	NaN	NaN	347082	NaN	B96 B98
freq	NaN	NaN	NaN	7	NaN	4
mean	29.699118	0.523008	0.381594	NaN	32.204208	NaN
std	14.526497	1.102743	0.806057	NaN	49.693429	NaN
min	0.420000	0.000000	0.000000	NaN	0.000000	NaN
25%	20.125000	0.000000	0.000000	NaN	7.910400	NaN
50%	28.000000	0.000000	0.000000	NaN	14.454200	NaN
75%	38.000000	1.000000	0.000000	NaN	31.000000	NaN
max	80.000000	8.000000	6.000000	NaN	512.329200	NaN

	Embarked
count	889
unique	3
top	S
freq	644
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN
75%	NaN
max	NaN

Summary of categorical attributes:

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3

top	Braund, Mr. Owen Harris	male	347082	B96	B98	S
freq		1	577	7	4	644

8) For multivariate statistics, you can compute the covariance and correlation between pairs of attributes.

```
In [9]: print('Covariance matrix:')
df.cov(numeric_only = True)
```

Covariance matrix:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	66231.000000	-0.626966	-7.561798	138.696504	-16.325843	-0.342697	161.883369
Survived	-0.626966	0.236772	-0.137703	-0.551296	-0.018954	0.032017	6.221787
Pclass	-7.561798	-0.137703	0.699015	-4.496004	0.076599	0.012429	-22.830196
Age	138.696504	-0.551296	-4.496004	211.019125	-4.163334	-2.344191	73.849030
SibSp	-16.325843	-0.018954	0.076599	-4.163334	1.216043	0.368739	8.748734
Parch	-0.342697	0.032017	0.012429	-2.344191	0.368739	0.649728	8.661052
Fare	161.883369	6.221787	-22.830196	73.849030	8.748734	8.661052	2469.436846

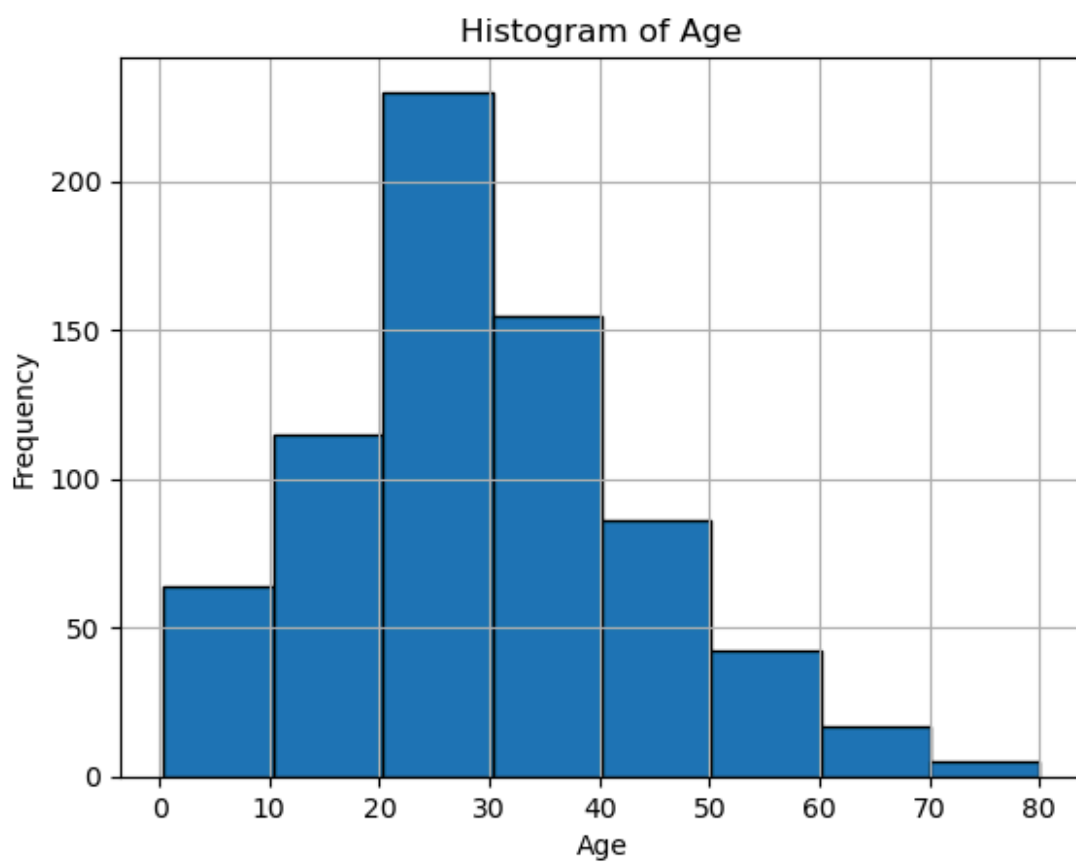
```
In [10]: print('Correlation matrix:')
df.corr(numeric_only = True)
```

Correlation matrix:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

9) Display the histogram for Age attribute by discretizing it into 8 separate bins and counting the frequency for each bin.

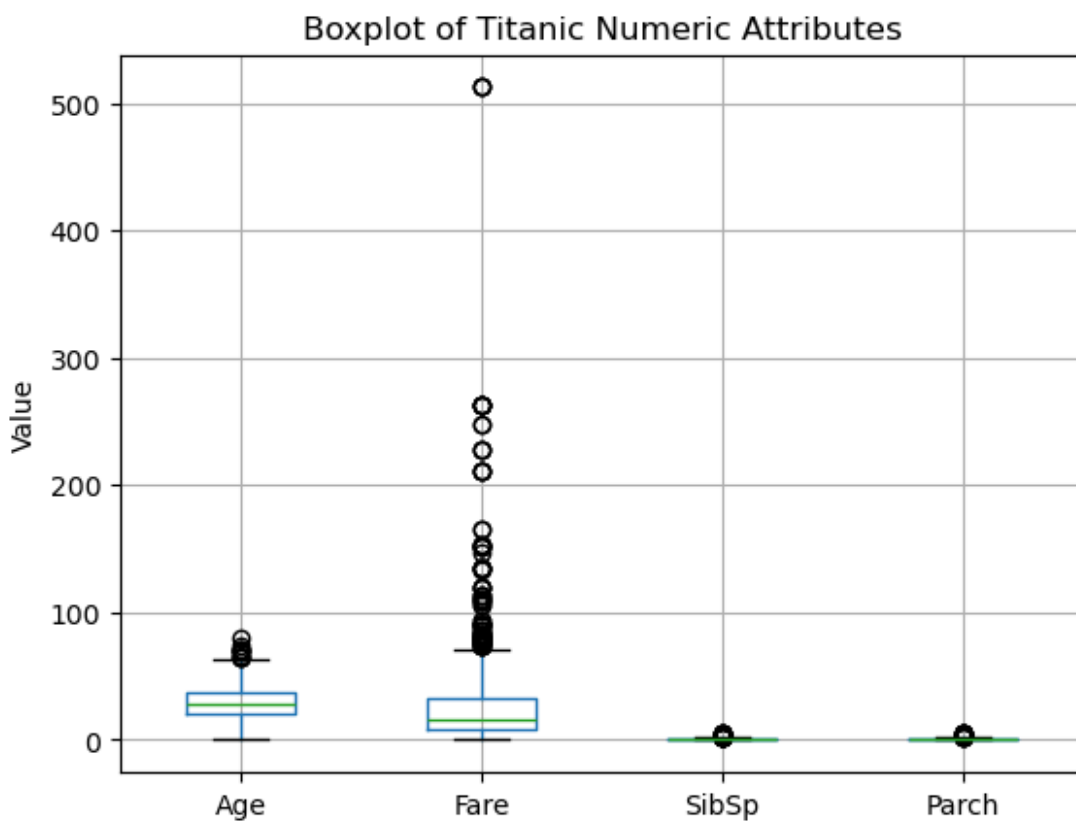
```
In [11]: df['Age'].dropna().hist(bins = 8, edgecolor = 'black', )
plt.title('Histogram of Age')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



10) A boxplot can also be used to show the distribution of values for each attribute.

```
In [12]: numeric = ['Age', 'Fare', 'SibSp', 'Parch']
data = df[numeric].dropna()

data.boxplot(column = numeric)
plt.title("Boxplot of Titanic Numeric Attributes")
plt.ylabel("Value")
plt.grid(True)
plt.show()
```

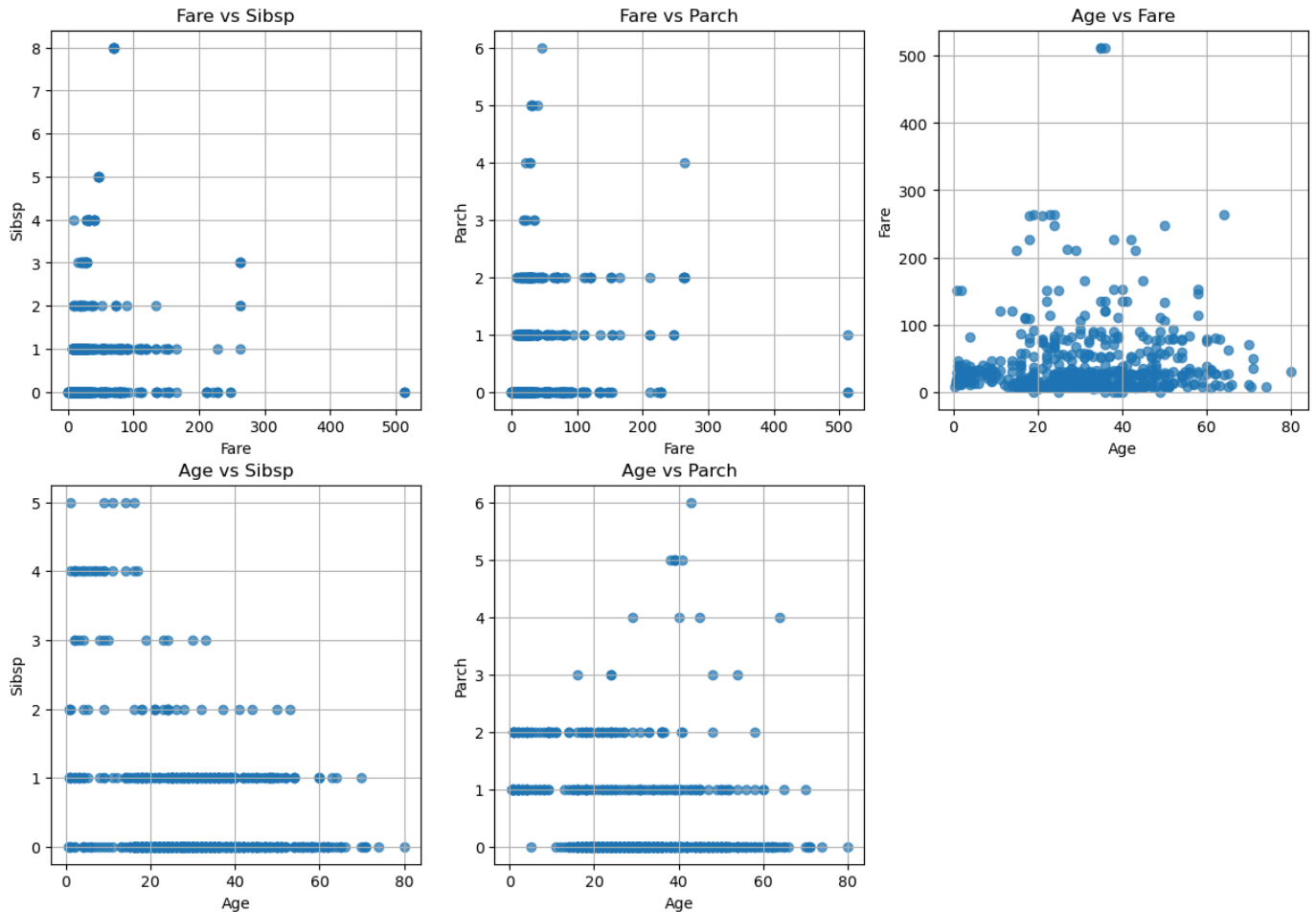


11) Display scatter plot for any 5 pair of attributes , we can use a scatter plot to visualize their joint distribution.

```
In [13]: pairs = [ ('Fare', 'SibSp'), ('Fare', 'Parch'), ('Age', 'Fare'), ('Age', 'SibSp'), ('Age', 'Parch')

plt.figure(figsize = (15, 10))

for i, (x, y) in enumerate(pairs):
    plt.subplot(2, 3, i + 1)
    plt.scatter(df[x], df[y], alpha=0.7)
    plt.xlabel(x.capitalize())
    plt.ylabel(y.capitalize())
    plt.title(f"{x.capitalize()} vs {y.capitalize()}")
    plt.grid(True)
```





Data Mining

Lab - 4

¥@\$# Kakadiya | 23010101121 |
25/6/2025

Step 1. Import the necessary libraries

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print(pd.__version__)
print(np.__version__)
```

2.2.2

1.26.4

Step 2. Import the dataset from this [address](#).

Step 3. Assign it to a variable called chipo.

```
In [2]: chipo = pd.read_csv("https://raw.githubusercontent.com/justmarkham/DAT8/master/data/chipotle.tsv")
chipo
```

Out[2]:

	order_id	quantity	item_name	choice_description	item_price
0	1	1	Chips and Fresh Tomato Salsa	NaN	\$2.39
1	1	1	Izze	[Clementine]	\$3.39
2	1	1	Nantucket Nectar	[Apple]	\$3.39
3	1	1	Chips and Tomatillo-Green Chili Salsa	NaN	\$2.39
4	2	2	Chicken Bowl	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	\$16.98
...	...	...	...	...	...
4617	1833	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Black Beans, Sour ...	\$11.75
4618	1833	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Sour Cream, Cheese...	\$11.75
4619	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Pinto...	\$11.25
4620	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Lettu...	\$8.75
4621	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Pinto...	\$8.75

4622 rows × 5 columns

Step 4. See the first 10 entries

In [3]:

chipo.head(10)

Out[3]:

	order_id	quantity	item_name	choice_description	item_price
0	1	1	Chips and Fresh Tomato Salsa	NaN	\$2.39
1	1	1	Izze	[Clementine]	\$3.39
2	1	1	Nantucket Nectar	[Apple]	\$3.39
3	1	1	Chips and Tomatillo-Green Chili Salsa	NaN	\$2.39
4	2	2	Chicken Bowl	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	\$16.98
5	3	1	Chicken Bowl	[Fresh Tomato Salsa (Mild), [Rice, Cheese, Sou...	\$10.98
6	3	1	Side of Chips	NaN	\$1.69
7	4	1	Steak Burrito	[Tomatillo Red Chili Salsa, [Fajita Vegetables...	\$11.75
8	4	1	Steak Soft Tacos	[Tomatillo Green Chili Salsa, [Pinto Beans, Ch...	\$9.25
9	5	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Black Beans, Pinto...	\$9.25

Step 5. What is the number of observations in the dataset?

```
In [4]: # Solution 1

chipo.shape[0]
```

Out[4]: 4622

```
In [5]: # Solution 2

chipo.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4622 entries, 0 to 4621
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              4622 non-null   int64
1   quantity              4622 non-null   int64
2   item_name             4622 non-null   object
3   choice_description     3376 non-null   object
4   item_price            4622 non-null   object
dtypes: int64(2), object(3)
memory usage: 180.7+ KB
```

Step 6. What is the number of columns in the dataset?

```
In [6]: chipo.shape[1]
```

Out[6]: 5

Step 7. Print the name of all the columns.

```
In [7]: chipo.columns
```

```
Out[7]: Index(['order_id', 'quantity', 'item_name', 'choice_description',
              'item_price'],
              dtype='object')
```

Step 8. How is the dataset indexed?

```
In [8]: chipo.index
```

Out[8]: RangeIndex(start=0, stop=4622, step=1)

Step 9. Number of Unique Items ?

```
In [9]: chipo['item_name'].nunique()
```

Out[9]: 50

Step 10. Which was the most-ordered item?

```
In [10]: chipo.groupby('item_name').sum().sort_values(['quantity'], ascending=False).head(1)
```

```
Out[10]:
```

	order_id	quantity	choice_description	item_price
item_name				
Chicken Bowl	713926	761	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	16.9810.98 11.258.75 8.4911.25 \$8.75 ...

Step 11. How many items were orderd in total?

```
In [11]: chipo['quantity'].sum()
```

```
Out[11]: 4972
```

Step 12. Turn the item price into a float

Step 12.a. Check the item price type

```
In [12]: chipo.dtypes['item_price']
```

```
Out[12]: dtype('O')
```

Step 12.b. Create a lambda function and change the type of item price

```
In [13]: dollarizer= lambda x:float(x[1:-1])  
chipo.item_price=chipo.item_price.apply(dollarizer)
```

Step 12.c. Check the item price type

```
In [14]: chipo.dtypes['item_price']  
chipo.item_price
```

```
Out[14]: 0      2.39  
1      3.39  
2      3.39  
3      2.39  
4     16.98  
...  
4617   11.75  
4618   11.75  
4619   11.25  
4620    8.75  
4621    8.75  
Name: item_price, Length: 4622, dtype: float64
```

Step 14. How much was the revenue for the period in the dataset?

```
In [15]: print('Revenue was: $'+str((chipo.item_price*chipo.quantity).sum()))
```

```
Revenue was: $39237.02
```

Step 15. How many orders were made ?

```
In [16]: chipo.order_id.nunique()
```

```
Out[16]: 1834
```

Step 17. How many different choice descriptions are there?

```
In [17]: chipo.choice_description.nunique()
```

```
Out[17]: 1043
```

Step 18. What items have been ordered more than 100 times?

```
In [18]: chipo.groupby('item_name').sum()['quantity'].loc[lambda x: x>100]
```

```
Out[18]: item_name
Bottled Water      211
Canned Soda        126
Canned Soft Drink  351
Chicken Bowl       761
Chicken Burrito    591
Chicken Salad Bowl 123
Chicken Soft Tacos 120
Chips              230
Chips and Fresh Tomato Salsa 130
Chips and Guacamole 506
Side of Chips      110
Steak Bowl         221
Steak Burrito      386
Name: quantity, dtype: int64
```

Step 19. What is the average revenue amount per order?

```
In [19]: # Solution 1

(chipo.item_price*chipo.quantity).sum()/chipo.order_id.nunique()
```

```
Out[19]: 21.39423118865867
```

```
In [20]: # Solution 2

chipo['revenue']=chipo.item_price*chipo.quantity
chipo.groupby('order_id').revenue.sum().mean()
```

```
Out[20]: 21.39423118865867
```



Data Mining

Lab - 5

¥@\$# Kakadiya | 23010101121 | 2/7/2025

Lab - 5 - Data Preprocessing

1) First, you need to read the titanic dataset from local disk and display Last five records

```
In [17]: import pandas as pd
```

```
In [18]: df=pd.read_csv('../data/titanic.csv')  
df
```

Out[18]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	

891 rows × 12 columns



In [19]:

df.tail()

Out[19]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarke
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	

2) Handle Missing Values in data set [use dropna(), fillna(), and interpolate]

In [20]:

```
# using dropna

df_dropna = df.dropna()
# df_dropna = df.dropna(how = 'all')
# df_dropna = df.dropna(how = 'any', axis = 1)
df_dropna
```

Out[20]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emba
	1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85
	3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123
	6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46
	10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.7000	G6
	11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103
	...	...	...	...	...	...	...	...	...	...	...	...
	871	872	1	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	11751	52.5542	D35
	872	873	0	1	Carlsson, Mr. Frans Olof	male	33.0	0	0	695	5.0000	B51 B53 B55
	879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.1583	C50
	887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42
	889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148

183 rows × 12 columns



```
In [21]: # using fillna
df_fillna=df.copy()
# df_fillna = df.fillna(30)
# df_fillna = df.fillna({'Age': 35, 'Cabin': 'C56'})
age_mean = df.Age.mean()
df_fillna = df.fillna({'Age':age_mean})
```

```
In [22]: # using interpolate

data_interpolate = df.interpolate()
```

```
data_interpolate['Age'] = data_interpolate['Age'].interpolate()  
data_interpolate
```

```
C:\Users\YASH\AppData\Local\Temp\ipykernel_19796\2446987545.py:3: FutureWarning: DataFrame.interpolate with object dtype is deprecated and will raise in a future version. Call obj.infer_objects(copy=False) before interpolating instead.
```

```
data_interpolate = df.interpolate()
```

Out[22]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emb
--	-------------	----------	--------	------	-----	-----	-------	-------	--------	------	-------	-----

0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	22.5	1	2	W./C. 6607	23.4500	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	

891 rows × 12 columns



3) Apply Scaling to AGE attribute with min max, decimal scaling and z score.

```
In [23]: scaling_df=df.copy()
scaling_df['Age']=scaling_df['Age'].interpolate()
df_min_max=scaling_df.copy()
mmAge=df_min_max['Age']
minAge=mmAge.min()
maxAge=mmAge.max()
df_min_max['MinMaxAge']=(mmAge-minAge)/(maxAge-minAge)
df_min_max
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Emb
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	22.5	1	2	W./C. 6607	23.4500	NaN	
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	

891 rows × 13 columns



Data Mining

Lab - 6

¥@\$# Kakadiya | 23010101121 | 9/7/2025

Dimensionality Reduction using NumPy

What is Data Reduction?

Data reduction refers to the process of reducing the amount of data that needs to be processed and stored, while preserving the essential patterns in the data.

Why do we reduce data?

- To reduce computational cost.
- To remove noise and redundant features.
- To improve model performance and training time.
- To visualize high-dimensional data in 2D or 3D.

Common data reduction techniques include:

- Principal Component Analysis (PCA)
- Feature selection
- Sampling

What is Principal Component Analysis (PCA)?

PCA is a **dimensionality reduction technique** that transforms a dataset into a new coordinate system. It identifies the **directions (principal components)** where the variance of the data is maximized.

Key Concepts:

- **Principal Components:** New features (linear combinations of original features) capturing most variance.
- **Eigenvectors & Eigenvalues:** Used to compute these principal directions.
- **Covariance Matrix:** Measures how features vary with each other.

PCA helps in **visualizing high-dimensional data**, **noise reduction**, and **speeding up algorithms**.

NumPy Functions Summary for PCA

Function	Purpose
<code>np.mean(X, axis=0)</code>	Compute mean of each column (feature-wise mean).
<code>X - np.mean(X, axis=0)</code>	Centering the data (zero mean).
<code>np.cov(X, rowvar=False)</code>	Compute covariance matrix for features.
<code>np.linalg.eigh(cov_mat)</code>	Get eigenvalues and eigenvectors (for symmetric matrices).
<code>np.argsort(values)[::-1]</code>	Sort values in descending order.
<code>np.dot(X, eigenvectors)</code>	Project original data onto new axes.

Step 1: Load the Iris Dataset

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: iris = pd.read_csv('../data/iris.csv')
iris
```

```
Out[2]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
In [3]: X = iris.drop(columns = "species")
y = iris['species'].map({
    'setosa': 0,
    'versicolor': 1,
    'virginica': 2
})
```

```
In [4]: print('Original shape:', X.shape)
```

Original shape: (150, 4)

Step 2: Standardize the data (zero mean)

```
In [5]: mean = np.mean(X, axis=0)
```

```
print("Mean of each feature:\n", mean)
```

Mean of each feature:

```
sepal_length    5.843333
sepal_width     3.057333
petal_length    3.758000
petal_width     1.199333
dtype: float64
```

```
In [6]: X_meaned = X - np.mean(X, axis = 0)
X_meaned.head()
```

```
Out[6]:
```

	sepal_length	sepal_width	petal_length	petal_width
0	-0.743333	0.442667	-2.358	-0.999333
1	-0.943333	-0.057333	-2.358	-0.999333
2	-1.143333	0.142667	-2.458	-0.999333
3	-1.243333	0.042667	-2.258	-0.999333
4	-0.843333	0.542667	-2.358	-0.999333

Step 3: Compute the Covariance Matrix

```
In [7]: cov_mat = np.cov(X_meaned, rowvar = False)
cov_mat.shape
```

```
Out[7]: (4, 4)
```

```
In [8]: print(cov_mat)

[[ 0.68569351 -0.042434  1.27431544  0.51627069]
 [-0.042434   0.18997942 -0.32965638 -0.12163937]
 [ 1.27431544 -0.32965638  3.11627785  1.2956094 ]
 [ 0.51627069 -0.12163937  1.2956094   0.58100626]]
```

Step 4: Compute eigenvalues and eigenvectors

```
In [9]: eigen_values, eigen_vectors = np.linalg.eigh(cov_mat)
print('Eigen Values:\n', eigen_values)
print('Eigen Vectors:\n', eigen_vectors)
```

Eigen Values:

```
[0.02383509 0.0782095  0.24267075 4.22824171]
```

Eigen Vectors:

```
[[ 0.31548719  0.58202985  0.65658877 -0.36138659]
 [-0.3197231  -0.59791083  0.73016143  0.08452251]
 [-0.47983899 -0.07623608 -0.17337266 -0.85667061]
 [ 0.75365743 -0.54583143 -0.07548102 -0.3582892  ]]
```

Step 5: Compute eigenvalues and eigenvectors

```
In [10]: sorted_index = np.argsort(eigen_values)[::-1]
sorted_eigenvalues = eigen_values[sorted_index]
sorted_eigenvectors = eigen_vectors[:, sorted_index]

print('Sorted index:\n', sorted_index)
print('Sorted Eigen Values:\n', sorted_eigenvalues)
print('Sorted Eigen Vectors:\n', sorted_eigenvectors)
```

```
Sorted index:
[3 2 1 0]
Sorted Eigen Values:
[4.22824171 0.24267075 0.0782095 0.02383509]
Sorted Eigen Vectors:
[[-0.36138659 0.65658877 0.58202985 0.31548719]
 [ 0.08452251 0.73016143 -0.59791083 -0.3197231 ]
 [-0.85667061 -0.17337266 -0.07623608 -0.47983899]
 [-0.3582892 -0.07548102 -0.54583143 0.75365743]]
```

Step 6: Select the top k eigenvectors (top 2)

```
In [11]: k = 2
eigenvector_subset = sorted_eigenvectors[:, 0:k]
print('Eigen Vector subset:\n', eigenvector_subset)
```

```
Eigen Vector subset:
[[-0.36138659 0.65658877]
 [ 0.08452251 0.73016143]
 [-0.85667061 -0.17337266]
 [-0.3582892 -0.07548102]]
```

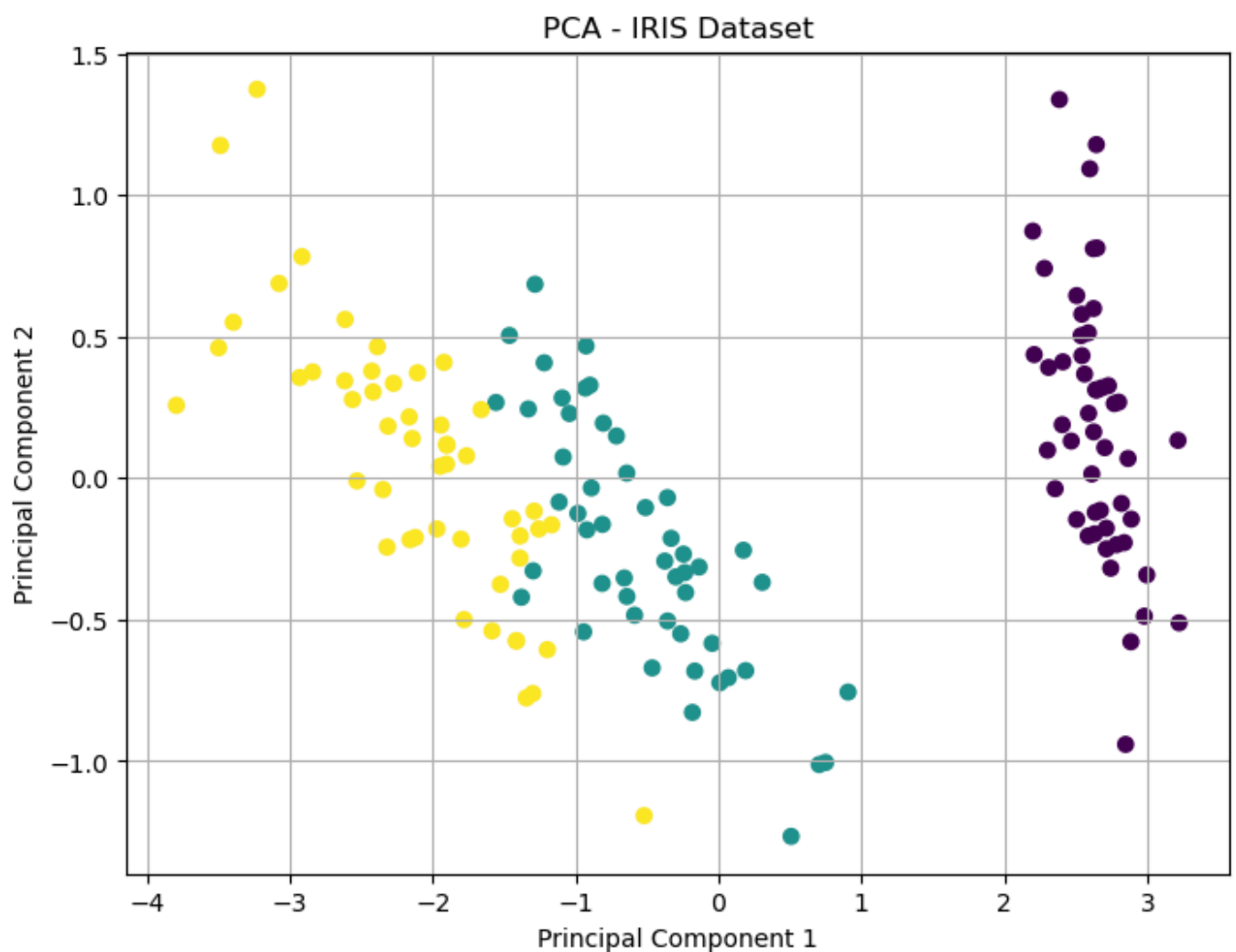
Step 7: Project the data onto the top k eigenvectors

```
In [12]: X_reduced = np.dot(X_meaned, eigenvector_subset)
print('Reduced data shape:', X_reduced.shape)
```

```
Reduced data shape: (150, 2)
```

Step 8: Plot the PCA-Reduced Data

```
In [13]: plt.figure(figsize = (8, 6))
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c = y)
plt.title('PCA - IRIS Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True)
plt.show()
```



Extra - Bining Method

5,10,11,13,15,35,50,55,72,92,204,215.

Partition them into three bins by each of the following methods: (a) equal-frequency (equal-depth) partitioning (b) equal-width partitioning

```
In [14]: data = [5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215]
data.sort()

print('Sorted Data:', data)

# (a) equal-frequency (equal-depth) partitioning
n = len(data)
k = 3
size = n // k

print('\n(a) Equal-Frequency Bins:')
for i in range(0, n, size):
    bin_data = data[i : i + size]
    print(f'Bin {i // size + 1}:', bin_data)

# (b) equal-width partitioning
min_val = min(data)
max_val = max(data)
range_val = max_val - min_val
width = range_val / k

bins = [[] for _ in range(k)]
```

```

for val in data:
    index = int((val - min_val) / width)
    if index == k: # edge case for the max value
        index -= 1
    bins[index].append(val)

print('\n(b) Equal-Width Bins:')
for i, b in enumerate(bins):
    bin_range_start = min_val + i * width
    bin_range_end = bin_range_start + width
    print(f'Bin {i + 1} ({bin_range_start:.2f} to {bin_range_end:.2f}): {b}')

```

Sorted Data: [5, 10, 11, 13, 15, 35, 50, 55, 72, 92, 204, 215]

(a) Equal-Frequency Bins:

Bin 1: [5, 10, 11, 13]

Bin 2: [15, 35, 50, 55]

Bin 3: [72, 92, 204, 215]

(b) Equal-Width Bins:

Bin 1 (5.00 to 75.00): [5, 10, 11, 13, 15, 35, 50, 55, 72]

Bin 2 (75.00 to 145.00): [92]

Bin 3 (145.00 to 215.00): [204, 215]

Data Mining

Lab - 7 (Part-2)

¥@\$# Kakadiya | 23010101121 |
30/7/2025

Step 1: Load the Dataset

Load the `Tdata.csv` file and display the first few rows.

```
In [3]: import pandas as pd
```

```
In [4]: df = pd.read_csv('../data/Tdata.csv')  
df
```

```
Out[4]:
```

	Transaction	bread	butter	coffee	eggs	jam	milk
0	T1	1	1	0	0	0	1
1	T2	1	1	0	0	1	0
2	T3	1	0	0	1	0	1
3	T4	1	1	0	0	0	1
4	T5	1	0	1	0	0	0
5	T6	0	0	1	1	1	0

Step 2: Drop the 'Transaction' Column

We're only interested in the items (not the transaction IDs).

```
In [5]: df_items = df.drop(columns = ['Transaction'])  
df_items
```

Out[5]:

	bread	butter	coffee	eggs	jam	milk
0	1	1	0	0	0	1
1	1	1	0	0	1	0
2	1	0	0	1	0	1
3	1	1	0	0	0	1
4	1	0	1	0	0	0
5	0	0	1	1	1	0

Step 3: Count Single Items

See how many transactions include each item.

In [6]: `df_items.sum()`

Out[6]:

```
bread      5
butter     3
coffee    2
eggs       2
jam        2
milk       3
dtype: int64
```

Step 4: Define Apriori Function

This function finds frequent itemsets of size 1, 2, and 3 with minimum support.

In [7]:

```
from itertools import combinations

def find_frequent_itemsets(df, min_support):
    n = len(df)
    result = []

    for k in [1, 2, 3]: #for 1-item, 2-item, 3-item sets
        for items in combinations(df.columns, k):
            mask = df[list(items)].all(axis = 1)
            support = mask.sum() / n

            if support >= min_support:
                result.append((frozenset(items), round(support, 2)))

    return result
```

Step 5: Run Apriori

Set `min_support = 0.6` and display the frequent itemsets.

In [8]:

```
frequent_itemsets = find_frequent_itemsets(df_items, 0.5)

for itemset, support in frequent_itemsets:
    print(f'{set(itemset)} → support: {support}')
```

```
{'bread'} → support: 0.83
{'butter'} → support: 0.5
{'milk'} → support: 0.5
{'butter', 'bread'} → support: 0.5
{'milk', 'bread'} → support: 0.5
```

Step 6 Display as a DataFrame

```
In [9]: result_df = pd.DataFrame(frequent_itemsets, columns = ['Itemset', 'Support'])
result_df
```

```
Out[9]:
```

	Itemset	Support
0	(bread)	0.83
1	(butter)	0.50
2	(milk)	0.50
3	(butter, bread)	0.50
4	(milk, bread)	0.50

Orange Tool : - >Generate Same Frequent Patterns in Orange tools

```
In [ ]:
```

Extra : - > Define Apriori Function without itertools

```
In [10]: def find_frequent_itemsets(df, min_support):
n = len(df)
result = []
columns = list(df.columns)

for i in range(len(columns)):
    items = [columns[i]]
    mask = df[items].all(axis=1)
    support = mask.sum() / n
    if support >= min_support:
        result.append((frozenset(items), round(support, 2)))

for i in range(len(columns)):
    for j in range(i + 1, len(columns)):
        items = [columns[i], columns[j]]
        mask = df[items].all(axis=1)
        support = mask.sum() / n
        if support >= min_support:
            result.append((frozenset(items), round(support, 2)))

for i in range(len(columns)):
    for j in range(i + 1, len(columns)):
        for k in range(j + 1, len(columns)):
            items = [columns[i], columns[j], columns[k]]
            mask = df[items].all(axis=1)
            support = mask.sum() / n
            if support >= min_support:
                result.append((frozenset(items), round(support, 2)))

return result
```

```
In [11]: frequent_itemsets = find_frequent_itemsets(df_items, 0.5)

result_df = pd.DataFrame(frequent_itemsets, columns = ['Itemset', 'Support'])
result_df
```

Out[11]:

	Itemset	Support
0	(bread)	0.83
1	(butter)	0.50
2	(milk)	0.50
3	(butter, bread)	0.50
4	(milk, bread)	0.50

Data Mining

Lab - 10

¥@\$# Kakadiya | 23010101121 |
20/8/2025

Implement Decision Tree(ID3) in python

Uses Information Gain to choose the best feature to split.

Recursively builds the tree until stopping conditions are met.

1. Calculate Entropy for the dataset.
2. Calculate Information Gain for each feature.
3. Choose the feature with maximum Information Gain.
4. Split dataset into subsets for that feature.
5. Repeat recursively until:

All samples in a node have the same label.

No features are left.

No data is left.

Step 2. Import the dataset from this [address](#).

import Pandas, Numpy

```
In [1]: import pandas as pd  
import numpy as np
```

Create Following Data

```
In [2]: data = pd.DataFrame({  
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny',  
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild',  
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal',  
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak', 'Weak',  
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes']  
})
```

In [3]: data

Out[3]:

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Now Define Function to Calculate Entropy

```
In [4]: def entropy(target_col):
        elements, counts = np.unique(target_col, return_counts=True)
        entropy = -np.sum([(counts[i]/np.sum(counts))*np.log2(counts[i]/np.sum(counts)) for i in range(len(counts))])
        return entropy
```

Testing of Above Function -

y = np.array(['Yes', 'No', 'Yes', 'Yes'])

Function Call - > entropy(y)

output - 0.8112781244591328

```
In [5]: y = np.array(['Yes', 'No', 'Yes', 'Yes'])
        print(entropy(y))
```

0.8112781244591328

Define function to Calculate Information Gain

```
In [6]: def information_gain(data, split_attribute_name, target_name):
        """
        Calculate the information gain of a dataset for a given attribute.
        """
        # Calculate the entropy of the whole dataset
        total_entropy = entropy(data[target_name])

        # Calculate the entropy of the dataset after splitting by the attribute
        values, counts = np.unique(data[split_attribute_name], return_counts=True)
        weighted_entropy = np.sum([(counts[i]/np.sum(counts))*entropy(data.where(data[split_attribute_name] == values[i])) for i in range(len(values))])

        return total_entropy - weighted_entropy
```

```
# Calculate the information gain
information_gain = total_entropy - weighted_entropy
return information_gain
```

Testing of Above Function-

```
data = pd.DataFrame({ 'Weather': ['Sunny', 'Sunny', 'Rain', 'Rain'], 'Play': ['Yes', 'No', 'Yes', 'Yes'] })
```

Function Call - > information_gain(data, 'Weather', 'Play')

Output - 0.31127812445913283

```
In [7]: data_test = pd.DataFrame({
'Weather': ['Sunny', 'Sunny', 'Rain', 'Rain'],
'Play':    ['Yes', 'No',   'Yes', 'Yes']
})

print(information_gain(data_test, 'Weather', 'Play'))
```

0.31127812445913283

Implement ID3 Algo

```
In [8]: def id3(data, features, target="PlayTennis"):
# If all labels are same → return the label
if len(np.unique(data[target])) == 1:
    return np.unique(data[target])[0]

# If no features left → return majority label
elif len(features) == 0:
    return np.unique(data[target])[np.argmax(np.unique(data[target], return_counts=True)[1])]

else:
    # Choose best feature
    item_values = [information_gain(data, feature, target) for feature in features]
    best_feature_index = np.argmax(item_values)
    best_feature = features[best_feature_index]

    # Create tree structure
    tree = {best_feature: {}}

    # Remove best feature from feature list
    features = [i for i in features if i != best_feature]

    # For each value of best feature → branch
    for value in np.unique(data[best_feature]):
        sub_data = data.where(data[best_feature] == value).dropna()
        subtree = id3(sub_data, features, target)
        tree[best_feature][value] = subtree

    return tree
```

Use ID3

```
In [9]: features = data.columns[:-1] # Exclude the target column
tree = id3(data, features)
```

Print Tree

```
In [10]: print(tree)
```

```
{ 'Outlook': { 'Overcast': 'Yes', 'Rain': { 'Wind': { 'Strong': 'No', 'Weak': 'Yes' } }, 'Sunny': { 'Humidity': { 'High': 'No', 'Normal': 'Yes' } } } }
```

Extra: Create Predict Function

```
In [11]: def predict(tree, sample):
          # If the node is a leaf node, return the label
          if not isinstance(tree, dict):
              return tree

          # Get the attribute to split on
          attribute = list(tree.keys())[0]
          attribute_value = sample[attribute]

          # Traverse the tree based on the attribute value
          if attribute_value in tree[attribute]:
              return predict(tree[attribute][attribute_value], sample)
          else:
              # Handle cases where the attribute value is not in the tree (e.g., return majority class
              # For simplicity, this implementation will return None or raise an error.
              # A more robust implementation would handle unknown values.
              return None # Or raise an error
```

Extra: Predict for a sample

sample = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}

Your Answer ?

```
In [12]: sample = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
```

```
In [13]: prediction = predict(tree, sample)
          print("Prediction:", prediction)
```

Prediction: No

Task

Complete the Python notebook to implement and test the ID3 decision tree algorithm, including functions for entropy, information gain, ID3 tree building, and prediction.

Complete entropy function

Subtask:

Implement the calculation of entropy in the provided entropy function.

Reasoning: Implement the entropy function based on the provided instructions to calculate the entropy of a given target variable.

Complete information_gain function

Subtask:

Implement the calculation of information gain in the provided information_gain function.